FastAPI: Comprehensive Theory and Concepts Study Guide

Module 1: FastAPI Basics

- **What is FastAPI?** FastAPI is a modern, fast (high-performance), web framework for building APIs with Python 3.8+ based on standard Python type hints. It is designed to be highly intuitive, easy to use, and production-ready.
- **Why FastAPI?**
  - **Performance**: On par with NodeJS and Go (thanks to Starlette and Uvicorn).
  - **Developer Speed**: Reduces code duplication and bugs, offering excellent autocompletion.
  - **Automated Docs**: Generates interactive OpenAPI documentations out-of-the-box.
- **Features & Advantages of FastAPI**
  - Automatic request data parsing and validation using Pydantic.
  - Native support for asynchronous programming (`async` / `await`).
  - Built-in dependency injection system.
  - Standards-based: Open standards (OpenAPI, JSON Schema) are used throughout.
- **FastAPI vs. Flask** Flask is a WSGI micro-framework, synchronous by default, requiring third-party libraries for validation (Marshmallow) and documentation (APISpec). FastAPI is ASGI-native, asynchronous, and validates data natively using Pydantic.
- **FastAPI vs. Django** Django is a "batteries-included" monolithic web framework with its own ORM and template engine, best for full-stack websites. FastAPI is a micro-framework optimized for building APIs, microservices, and serving machine learning models.
- **FastAPI Architecture**
  - **ASGI (Asynchronous Server Gateway Interface)**: The standard interface for asynchronous Python web apps, allowing concurrent connection handling (WebSockets, long polling). Replaces **WSGI**, which is synchronous and blocks threads per request.
  - **Uvicorn**: A lightning-fast ASGI web server implementation used to run FastAPI applications.
  - **Starlette**: A lightweight ASGI toolkit that FastAPI inherits from for routing, web request handling, and WebSocket support.

Module 2: API Basics

- **What is a REST API?** Representational State Transfer (REST) is an architectural style for designing networked applications, using standard stateless HTTP operations.
- **REST Principles**
    1. **Client-Server Architecture**: Separates user interface concerns from data storage concerns.
    2. **Statelessness**: Each request must contain all the information necessary to process it. No session state is stored on the server.
    3. **Cacheability**: Server responses must define themselves as cacheable or non-cacheable.
    4. **Uniform Interface**: Resources are identified by URIs and manipulated using standard HTTP methods.
    5. **Layered System**: Clients cannot tell if they are connected directly to the end server or an intermediate proxy/load balancer.
- **HTTP Methods**
    - **GET**: Retrieves data from the server. Safe and idempotent.
    - **POST**: Sends data to the server to create a new resource. Neither safe nor idempotent.
    - **PUT**: Replaces an existing resource or creates it if missing. Idempotent.
    - **PATCH**: Applies partial updates to an existing resource. Non-idempotent in standard REST.
    - **DELETE**: Removes a specific resource from the server. Idempotent.
- **Safe Methods**: HTTP methods that do not modify database or server state (e.g. `GET`, `HEAD`, `OPTIONS`).
- **Idempotent Methods**: HTTP methods where executing a request multiple times yields the exact same server/resource state as executing it once (e.g. `GET`, `PUT`, `DELETE`).

Module 3: Routing & Parameters

- **Path Parameters** Variables embedded directly inside the URL path. They are mandatory: `python @app.get("/users/{user_id}") def read_user(user_id: int): return {"user_id": user_id}`
- **Query Parameters** Optional key-value pairs appended after the `?` character in the URL. FastAPI infers parameters as query parameters if they are not defined in the path: `python @app.get("/items/") def read_items(limit: int = 10, skip: int = 0): return {"limit": limit, "skip": skip}`
- **Request Body** To receive JSON data, you define a schema using Pydantic's `BaseModel`. FastAPI parses the JSON body and injects it as an object parameter.

- **Response Model** Defined in the path operation decorator. Filters out private fields (e.g., passwords) and validates output: `python @app.get("/users/{user_id}", response_model=UserResponse)`
- **APIRouter** A tool to split large APIs into multiple modular files. Allows setting a common **Route Prefix** and **Tags** for documentation: `python router = APIRouter(prefix="/items", tags=["items"])`
- **API Versioning & Documentation** FastAPI generates interactive documentations automatically:
  - **Swagger UI** at `/docs` (interactive UI).
  - **ReDoc** at `/redoc` (structured documentation UI).

Module 4: Pydantic

- **What is Pydantic?** A data validation and settings management library based on Python type hints. It enforces type hints at runtime, parsing inputs and outputting clean validation errors.
- **Key Features**
  - **BaseModel**: The base class for defining schemas.
  - **Field Validation**: Adding constraints (e.g., character length, numerical range) using `Field()`: `python class User(BaseModel): username: str = Field(min_length=3, max_length=50) age: int = Field(gt=0, lt=120)`
  - **Nested Models**: Models containing other Pydantic models.
  - **Custom Validators**: Writing custom checks using `@field_validator` decorators: `python @field_validator("email") def validate_email(cls, v): if "@" not in v: raise ValueError("Invalid email address") return v`
  - **Serialization & Deserialization**: Transforming models to dictionaries/JSON (`model_dump()`, `model_dump_json()`) or creating models from raw inputs (`model_validate()`).

Module 5: Dependency Injection

- **Dependency Injection (DI)** A programming design pattern where a function declares the resources (dependencies) it needs rather than instantiating them itself. In FastAPI, this is done using **`Depends()`**. ```python def get_db(): db = SessionLocal() try: yield db finally: db.close()

  @app.get("/items/") def read_items(db: Session = Depends(get_db)): return db.query(Item).all() `` * **Why use Dependency Injection?** * **Code Reuse**: Shared connection pools or authentication checkers. * **Simplified Testing**: Easily override dependencies (e.g., replacing the live DB dependency with a mock database for tests).

`* **Resource Lifecycle Management**: Safe closing of resources (like DB sessions) using yield`.

Module 6: Middleware

- **What is Middleware?** A function that intercepts every HTTP request before it is handled by a route, and intercepts every response before it is returned to the client.
- **Core Concepts**
  - **CORS Middleware**: Web browsers restrict cross-origin requests. FastAPI requires configuring `CORSMiddleware` to whitelist origins (domains, ports) that are allowed to call the API: `python app.add_middleware( CORSMiddleware, allow_origins=["http://localhost:3000"], allow_credentials=True, allow_methods=["*"], allow_headers=["*"], )`
  - **Logging & Custom Middleware**: Modifying request headers, capturing processing durations, or logging client IPs.

Module 7: Authentication & Authorization

- **JWT (JSON Web Token)** A compact, URL-safe standard (`RFC 7519`) representing claims transferred between two parties.
  - *Payload*: Contains user identifiers (e.g., `sub`), expiration times (`exp`), and roles.
  - *Mechanism*: Server signs the token using a secret key. The client passes this token in the `Authorization: Bearer <token>` header. The server decodes it stateless without database queries.
- **OAuth2**: An authorization framework. FastAPI includes utilities to handle OAuth2 password grant flows (`OAuth2PasswordBearer`).
- **Password Hashing**: Storing passwords securely using hashing algorithms like **bcrypt** or **argon2**. Passwords should never be stored in plain text.
- **RBAC (Role-Based Access Control)**: Restricting API route execution based on user groups or claims decoded from JWTs.

Module 8: Database Integration

- **SQLAlchemy**: A popular Object-Relational Mapper (ORM) for Python. Maps Python classes to database tables.
- **Async SQLAlchemy**: Supported in SQLAlchemy 1.4/2.0+, allowing database queries to execute asynchronously using `asyncio` drivers (like `asyncpg`), bypassing blocking thread calls.

- **Alembic**: A lightweight database migration tool for SQLAlchemy. It tracks schema modifications in Python scripts and manages upgrades/downgrades on target databases.
- **Session Management**: Using `scoped_session` or Dependency Injection with `yield` to ensure database connections are opened on-request and closed cleanly when the request ends.

Module 9: Async Programming in FastAPI

- **async & await** Keywords representing asynchronous execution. `async def` tells Python that the function is a coroutine and can yield control to the **Event Loop** when encountering I/O operations using the `await` keyword.
- **When to use `async def` vs. `def`?**
  - Use `async def` when performing non-blocking I/O operations (e.g., querying external APIs, calling async databases).
  - Use standard `def` when executing CPU-bound calculations or using blocking libraries (like standard SQL Alchemy). FastAPI runs blocking `def` functions in an external thread pool to prevent blocking the event loop.
- **Background Tasks**: Built-in utility to run operations *after* returning an HTTP response, freeing up API response times: `python @app.post("/send-email/") def send_email(email: str, background_tasks: BackgroundTasks): background_tasks.add_task(email_worker_function, email) return {"message": "Email is being sent in the background"}`

Module 10: Production & Best Practices

- **Exception Handling**: Custom exception handlers can be registered using `@app.exception_handler(CustomException)`.
- **Validation Errors**: Overriding default `RequestValidationError` to return custom formatted JSON errors.
- **Environment Variables**: Utilizing `pydantic-settings` (`BaseSettings`) to load environment variables from `.env` files with strict type validations.
- **Deployments**
  - **Gunicorn + Uvicorn**: Gunicorn acts as a process manager monitoring worker processes, while Uvicorn workers handle the ASGI requests.
  - **Nginx**: Acts as a reverse proxy, SSL termination layer, and static asset cache in front of the application server.
- **Advanced Features**:
  - **File Uploads**: Using `UploadFile` (spolls to disk for large files) vs `File` (loads file completely in memory).

- **WebSockets**: Bidirectional real-time persistent connections.
- **Redis Caching**: Caching expensive database lookups in Redis to optimize latency.

⭐ High-Yield Interview Questions (FastAPI)

1. **What is FastAPI and what are its key features?** An ASGI-based web framework for building APIs with Python type hints. Key features: speed (Uvicorn/Starlette), auto-documentation (Swagger/ReDoc), input validation (Pydantic), and native async support.
2. **FastAPI vs Flask?** Flask is a synchronous WSGI framework that requires third-party plugins for validation and documentation. FastAPI is an asynchronous ASGI framework with native validation (Pydantic) and auto-documentation.
3. **What is ASGI and how does it differ from WSGI?** WSGI (Web Server Gateway Interface) is synchronous and blocks threads per request. ASGI (Asynchronous Server Gateway Interface) is asynchronous and supports concurrent connections, WebSockets, and non-blocking I/O.
4. **How does FastAPI handle data validation?** FastAPI parses JSON bodies and validates input constraints using Pydantic schemas. If validation fails, it automatically returns a `422 Unprocessable Entity` JSON response.
5. **Explain Dependency Injection in FastAPI.** Declared using `Depends()`. It allows route handlers to declare reusable requirements (database sessions, authentication). It simplifies code reuse and test mocking.
6. **How do you handle JWT authentication in FastAPI?** The client logs in, and the server returns a signed JWT. The client passes this token in the `Authorization: Bearer <token>` header. The API decodes the token statefully or statelessly using security dependencies.
7. **When should you write `async def` vs. `def` in route handlers?** Use `async def` if you are calling asynchronous libraries with `await` (non-blocking I/O). Use standard `def` for CPU-bound computations or blocking database calls. FastAPI runs standard `def` functions in a separate thread pool to prevent blocking the main event loop.
8. **What is the role of the `response_model` parameter?** It is defined in the path decorator to filter output fields, format output data, and validate response schemas before sending data to clients.
9. **How do you handle CORS issues in FastAPI?** By importing and adding `CORSMiddleware` to the FastAPI application, defining whitelisted origins, methods, and headers.
10. **How do you deploy a FastAPI app to production?** Use a Docker container. Run a process manager like Gunicorn with Uvicorn worker classes (`gunicorn -k uvicorn.workers.UvicornWorker`) behind an Nginx reverse proxy.

📚 Module Summary & Key Takeaways

> **Theory Focus:** ASGI vs WSGI, Pydantic Serialization, Dependency Injection, and JWT Security.

This module details asynchronous microservices and API integration using FastAPI. Key operational details include:

- **Asynchronous Execution (ASGI):** By using an Asynchronous Server Gateway Interface (ASGI) native stack (Uvicorn and Starlette), FastAPI processes concurrent persistent connections (WebSockets, streaming) efficiently on an event loop, bypassing synchronous thread-per-request blocking.
- **Validation & Pydantic:** Enforces type safety natively using Python type hints, compiling data parsing and strict payload validation through **Pydantic** to catch errors before code execution.
- **DI & Authentication:** A modular dependency injection system (via `Depends`) manages database sessions, parameters, and security hooks like OAuth2 flow checks and JSON Web Token (JWT) token verification.